

RAG: архитектура, метрики, production



Дмитрий

Главный аналитик-исследователь,
Клиентский сервис, Газпромбанк

Кто я и почему занимаюсь RAG-системами?

Senior Data Scientist LLM

Занимаюсь разработкой RAG-системы для Контактного Цетра Газпромбанка. Преподаю в МГТУ им.Н.Э. Баумана, 444, иногда в МГИМО, МФТИ и Сириусе.



Что такое **RAG (Retrieval Augmented Generation)**?

Retrieval - обычный поиск в браузере, возвращает релевантные ссылки.

Их нужно открывать 🕒

Generation - поиск в ChatGPT/Claude, возвращает конкретные ответы.

Их нужно проверять 🕒

RAG находит релевантные ссылки и дает на вход нейросети вместе с вопросом. Так **быстрее и точнее**.



Правильно ли GPT подскажет расписание экзаменов? а RAG?*

*Подсказка: A. Korzybski The map is not a territory. Indexes, Dates, Et Cetera.

Парсинг источников данных

Индексация (offline)



Поиск (online, на каждый запрос)



Генерация



Обратная связь



1. API (лучший способ)
2. Playwright (динамика)
3. Chrome MCP (агенты)
4. Scrapy (быстро)
5. Crawl4AI (сразу в MD)
6. Firecrawl (платно)
7. ~~Selenium (медленно)~~
8. ~~BeautifulSoup (статика)~~

<https://books.toscrape.com/>

FULLKIT* - все необходимое для реализации RAG.

*Eliyahu M. Goldratt The Goal: A Process of Ongoing Improvement

Парсинг документов PDF, DOCX, ...

Индексация (offline)



Поиск (online, на каждый запрос)



Генерация



Обратная связь



1. MinerU2.5-Pro (16Gb)
2. PaddleOCR-VL-1.5 (4Gb)
3. LightOnOCR-2 (таблицы)
4. Docling (ERC-2)
5. Marker (SuryaOCR)

VLM лучше классики.

VLM finetune лучше VLM.

Таблицы - отдельная задача.

Таблицы в JSON (не MD).

Каждый новый документ в базе - это запуск OCR.

Trade-off: качество и GPU-часы на 1000 страниц.

Разделение документов на чанки

Индексация (offline)



Поиск (online, на каждый запрос)



Генерация



Обратная связь



1. Recursive (по знакам препинания)
2. Structure-Aware (#, <h1>, <h2>, ...)
3. Late chunking (длинные документы)
4. ~~Semantic chunking (3%)~~
5. Contextual retrieval (максимальное качество для VIP-документов)
6. GraphRAG (ответ в разных документах)
7. Adaptive chunking

5 levels of text splitting

Модели переводящие чанки текста в векторы

Индексация (offline)



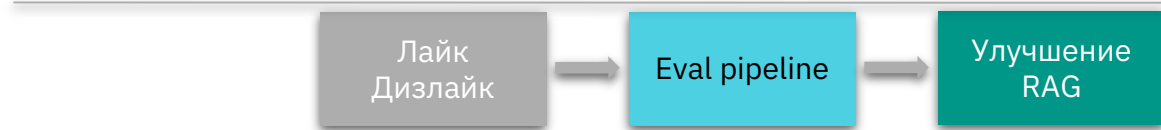
Поиск (online, на каждый запрос)



Генерация



Обратная связь



1. Qwen3-emb-0.6b (1.5Gb)
2. Qwen3-emb-4b (9Gb)
3. ...

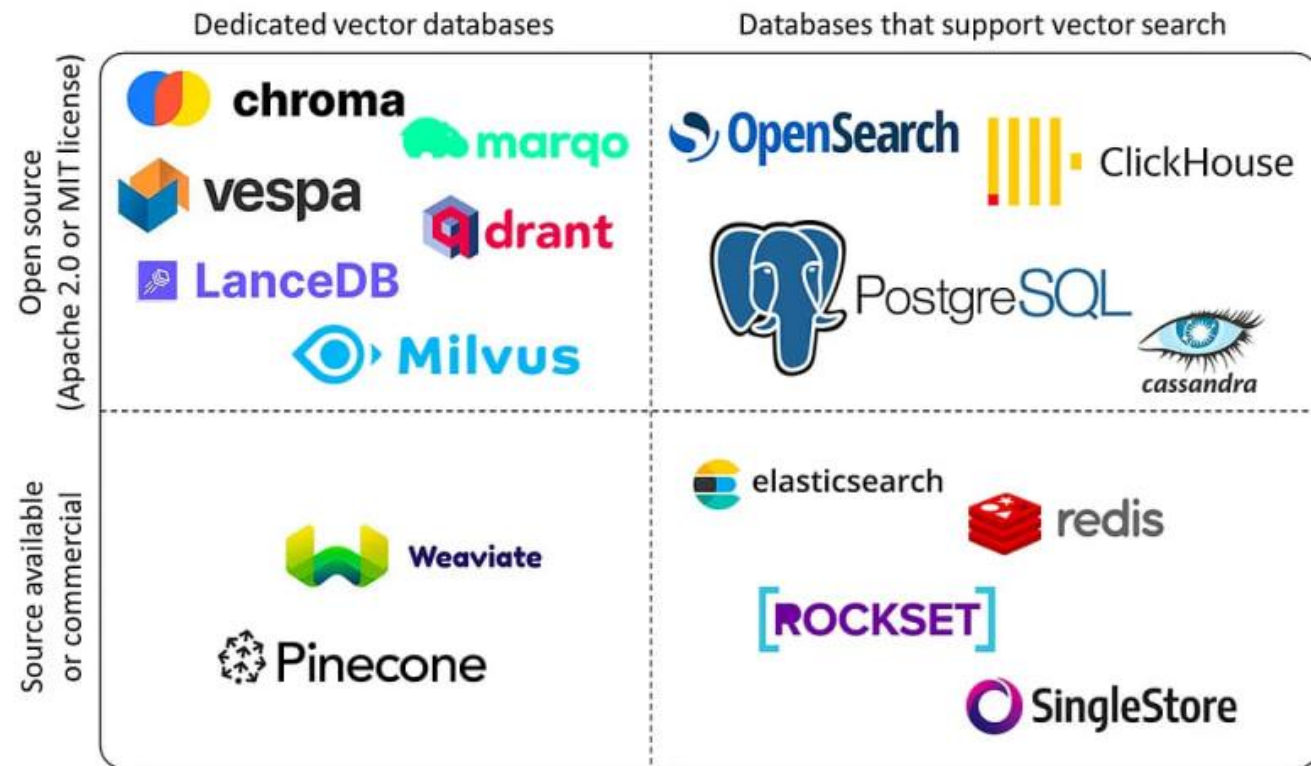
Finetune retrieval*

<https://habr.com/ru/companies/sberdevices/articles/831150/>

<https://mera.ai.ru/ru/text/leaderboard>
<https://superlinked.com/vector-db-comparison>
<https://ann-benchmarks.com/>

*Если терминология домена специфична. Например, медицинские термины.

Векторные базы данных



1. Milvus(>100М, K8s, RRF, лучшее масштабирование)
2. Qdrant (>100М, <5ms, лучшая фильтрация)
3. PostgreSQL (Supabase) (<50М, развернут везде)
4. Aerospike (>1B, лучшая по скорости)
5. Faiss (IDMap2)
6. Weaviate, Chroma

Источник изображения: The 5 Best Vector Databases | A List With Examples | DataCamp

Гибридный поиск

Индексация (offline)



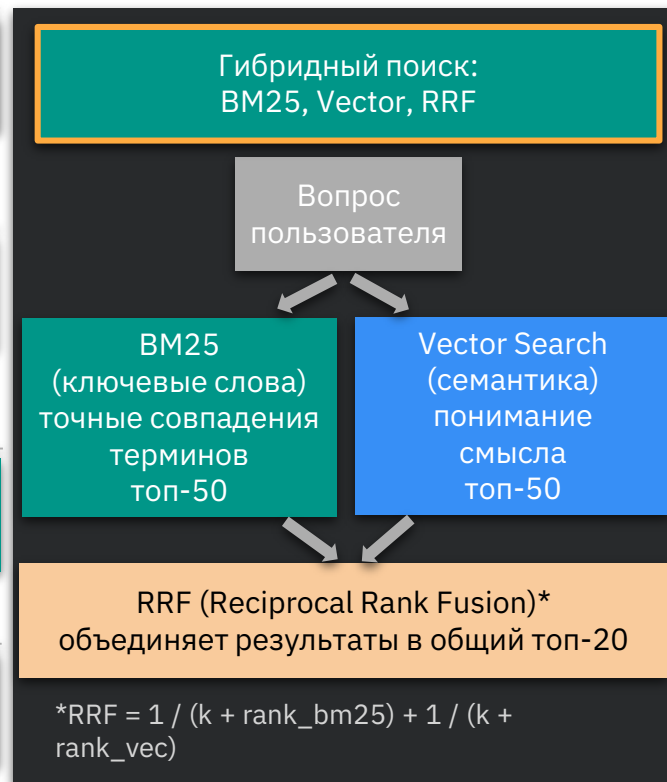
Поиск (online, на каждый запрос)



Генерация



Обратная связь



Реранкинг

Индексация (offline)



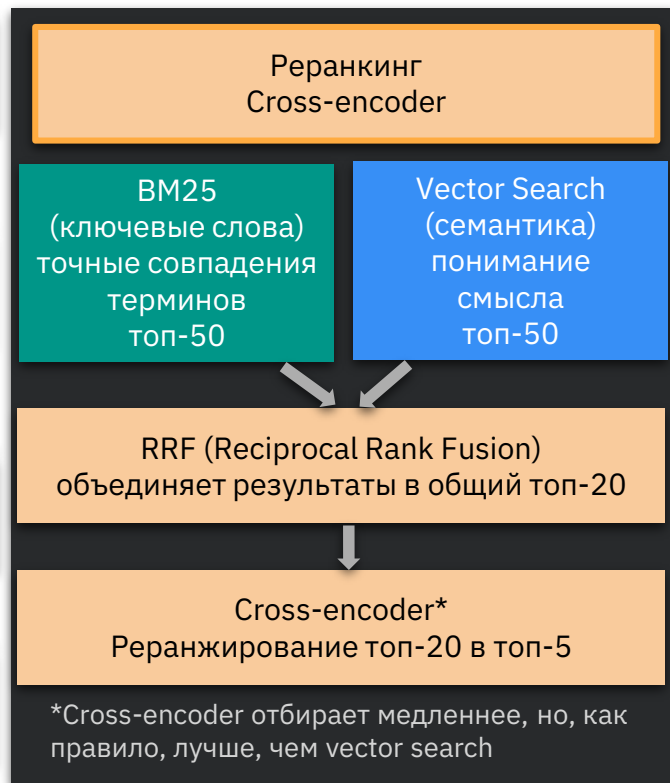
Поиск (online, на каждый запрос)



Генерация



Обратная связь



Улучшение промптов: SGR и SO

```
from pydantic import BaseModel
from typing import List, Literal
from openai import OpenAI

client = OpenAI()

# SGR
class Step(BaseModel):
    step: Literal[1, 2, 3, 4, 5]
    action: Literal[
        "зафиксировать_критерий_качества",
        "разложить_RAG_на_подходы",
        "собрать_свежие_работы_arxiv",
        "выделить_ключевые_идеи",
        "оценить_прирост_и_сравнить"
    ]
    description: str
    result: str
```

```
# SO
class Paper(BaseModel):
    technique: str
    paper: str
    date: str
    metric_gain: str
    metric: str

class Output(BaseModel):
    steps: List[Step]
    top_papers: List[Paper]

resp = client.responses.create(
    model="gpt-4.1",
    response_format=Output,
    input="Какая RAG-техника даёт
    максимальный прирост точности?"
)
print(resp.output_parsed)
```

```
# JSON
{
    "steps": [{
        "step": 1,
        "action": "зафиксировать
        критерий качества",
        "description": "выбор метрик",
        "result": "Recall@5, MRR"},
        ...],
    "top_papers": [{
        "technique": "TaSR-RAG",
        "paper": "Taxonomy RAG",
        "date": "2026",
        "metric_gain": "+14%",
        "metric": "Recall@5"},
        ...]
}
```

SGR (Schema guided reasoning) - подробное описание инструкции для LLM.

SO (Sstructured Output) - описание строгого формата ответа в виде JSON-схемы

https://github.com/NirDiamant/Prompt_Engineering

RAG-техники

Agentic RAG / GraphRAG / Contextual Retrieval / Late Chunking

Context Compression / Multi-stage Reranking / Query Expansion / Multi-hop / Parent Documents

Как работает Agentic RAG?

Автономные агенты управляют retrieval-циклом: планирование, рефлексия, tool use, multi-agent

Архитектура

1. Query Planner

Декомпозиция запроса на подзадачи

2. Retrieval Agent

Динамический выбор источника и стратегии

3. Reflection / Self-critique

LLM-as-Judge проверяет полноту ответа

4. Iterative Loop

Повторный поиск при недостатке данных

Когда применять

Error rate -78%

- **Multi-hop вопросы**
Связь фактов из разных документов
- **Cross-system retrieval**
БД + API + vector store одновременно
- **Сложная аналитика**
Финансы, юридические, медицинские кейсы
- **Задачи с планированием**
Где single-shot retrieval недостаточен

```
# Agentic RAG: LangGraph self-correcting loop
from langgraph.graph import StateGraph

def retrieval_agent(state):
    docs = retrieve(state['query'])
    if judge_relevance(docs, state['query']) < 0.7:
        state['query'] = rewrite_query(state['query'])
        return 'retrieve' # loop back for better results
    return 'generate' # proceed to LLM generation
```

Пример Agentic RAG в production

Алгоритм работы

- 1 Создается набор инструментов (промпты, python-функции, API)
- 2 Пользователь вводит запрос
- 3 AI-агент маршрутизирует запрос к инструменту
- 4 AI-агент может преобразовать запрос
- 5 AI-агент получает результаты (возможно итеративно улучшая)
- 6 AI-агент преобразует результат и выводит пользователю

```
- DB CALL: list_tables  
- DB CALL: describe_table  
- DB CALL: execute_query
```

The cheapest product is the Mouse, costing \$29.99.

Архитектура Gen AI Agent

User
Input

Function
Call

Function
Response

Model
Output

```
%sql  
CREATE TABLE IF NOT EXISTS  
products (  
    product_id INTEGER PRIMARY  
KEY,  
    product_name VARCHAR(255),  
    price DECIMAL(10, 2)  
);  
INSERT INTO products VALUES  
('Laptop', 799.99),  
('Keyboard', 129.99),  
('Mouse', 29.99);
```

```
def list_tables() -> list[str]:  
def describe_table(name) -> list:  
def execute_query(sql) -> list:  
  
db_tools = [list_tables,  
            describe_table, execute_query]  
model = genai.GenerativeModel(  
    'gemini-1.5-flash',  
    tools=db_tools)  
  
resp = chat.send_message(  
    'Какой продукт самый дешевый?')
```

Уровень агентности AI-агентов

Уровень	Описание	Паттерн	Фреймворк	Код
LLM	Вывод LLM не влияет на поток выполнения программы	Простой процессор	-	<code>process_llm_output(response)</code>
LLM + планирование	Вывод LLM определяет базовый поток управления	Маршрутизатор (Router)	LangChain	<code>if llm_decision(): path_a() else: path_b()</code>
LLM + планирование + инструменты	Вывод LLM определяет выполнение функций	Вызов инструмента (Tool call)	FastMCP	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
LLM + планирование + инструменты + граф шагов	Вывод LLM управляет итерацией и продолжением программы	Многошаговый агент	LangGraph	<code>while should_continue(): execute_next_step()</code>
LLM + планирование + инструменты + граф шагов + вложенные графы	Один агентный workflow может запустить другой агентный workflow	Мульти-агентная система	LangGraph	<code>if llm_trigger(): execute_agent()</code>

Материалы по Agentic RAG

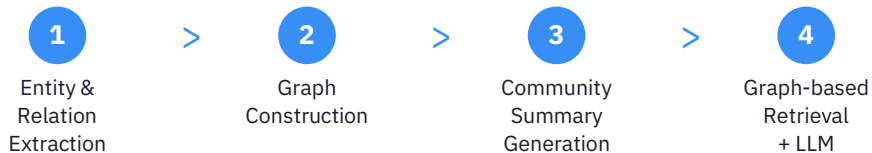
- 1 Introduction to Agents**
[Google AI](#)
- 2 Yandex AI Studio Series**
yandex.cloud/ru/ai-studio-series
- 3 Smolagents**
huggingface.co/blog/smolagents
- 4 5-Day GenAI Intensive Course**
kaggle.com/learn-guide/5-day-genai
- 5 5-Day GenAI with Google (GitHub)**
github.com/sauravkumar8178/5-Day-Gen-AI-Intensive-Course-with-Google
- 6 Agentic AI Hands-on**
cognitiveclass.ai/learn/agent-ai-hands-on
- 7 Agentic RAG Course (Stepik)**
stepik.org/course/250355/promo

“До действительно умных AI-агентов нам еще лет 10”

GraphRAG

Knowledge Graph + RAG: связи между сущностями вместо изолированных чанков

Pipeline (Microsoft GraphRAG)



Когда применять

- Multi-hop запросы между связанными сущностями
- Global questions: "Какие основные темы?"
- Enterprise KB с иерархией
- Финансы, медицина, право

```
# GraphRAG: Neo4j + LLM retrieval
from graphrag import GraphRAGPipeline

pipeline = GraphRAGPipeline(
    llm=llm, graph_db=Neo4jDriver(uri, auth),
    community_algo='leiden',
    entity_types=['Person', 'Org', 'Event']
)
pipeline.index(documents) # build KG

result = pipeline.query(
    'How are companies X and Y related?',
    search_type='global' # or 'local'
)
```

Метрики (benchmarks)

Accuracy: 80% vs 50%

на global queries (Lettria/AWS)

Comprehensiveness: +72-83%

Microsoft Research 2024

Factual accuracy: +23%

e-Commerce QA (MS Research)

LazyGraphRAG: -99.9% cost

индексации при том же качестве

Contextual Retrieval

LLM добавляет контекст документа к каждому чанку перед эмбедингом

Проблема

Чанк "Its more than 3.85 million inhabitants..." теряет связь с "Berlin" из соседнего чанка. Embedding не знает, что "Its" = Berlin. Результат: retrieval failure на запрос "население Берлина".

Решение: 3 шага

1. Chunk

Разбиваем документ на чанки

2. Contextualize

LLM генерирует контекст из документа

3. Embed [context + chunk]

Embedding включает global context

```
# Contextual Retrieval pipeline
def contextualize_chunk(doc: str, chunk: str) -> str:
    prompt = f"""
    Document: {doc[:2000]}
    Chunk: {chunk}
    Give a short context (2-3 sentences)
    that situates this chunk within the doc.
    """

    context = llm.generate(prompt)
    return f"[CONTEXT: {context}] {chunk}"

# Embed enriched chunks
for chunk in chunks:
    enriched = contextualize_chunk(full_doc, chunk)
    embedding = embed_model.encode(enriched)
    vector_db.upsert(embedding, metadata=chunk)
```

Влияние на метрики

Hit Rate: +15-20%

на анафорических запросах

Семантическая когерентность

лучше, чем late chunking
(ECIR 2025 benchmark)

Trade-off:

Доп. LLM-вызов на каждый чанк.
prompt caching для снижения стоимости.

Late Chunking & Context Compression

Late Chunking (Jina AI)

Идея:

Сначала прогоняем весь документ через long-context embedding model, затем делим на чанки с span pooling.

Каждый токен уже "видел" весь контекст через self-attention. Нет LLM-вызовов.

Retrieval accuracy +10-12%

на документах с анафорами

```
# Late Chunking: Jina Embeddings
from jina_embeddings import JinaModel

model = JinaModel('jina-embeddings-v3')

# 1. Encode full document
token_embeddings = model.encode_tokens(
    full_document # up to 8192 tokens
)

# 2. Pool by chunk boundaries
chunks = split_by_sentences(full_document)
chunk_embeds = span_pool(
    token_embeddings, chunk_boundaries
)
```

Context Compression

-35% токенов

Идея:

Сжимаем извлеченные чанки, оставляя только релевантную запросу информацию. Уменьшаем шум в контексте LLM.

Методы:

LLMLingua, Selective Context, Extractive compression (sentence-level)

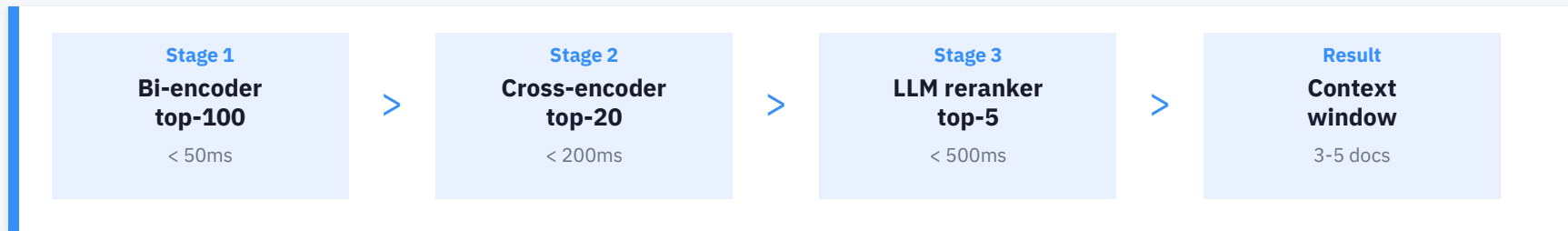
Token reduction: -35-50%

при сохранении Faithfulness > 0.85

```
# Context Compression
from langchain.retrievers import (
    ContextualCompressionRetriever
)
from langchain.compressors import (
    LLMChainExtractor
)
compressor = LLMChainExtractor(
    llm=llm
)
retriever = ContextualCompressionRetriever(
    base_retriever=vector_retriever,
    base_compressor=compressor
)
```

Multi-stage Reranking

Bi-encoder (fast, recall) + Cross-encoder (slow, precision): каскад для точного ранжирования



```
# Multi-stage reranking pipeline
from sentence_transformers import CrossEncoder

# Stage 1: Bi-encoder retrieval (fast)
candidates = vector_db.search(query_embedding, top_k=100)

# Stage 2: Cross-encoder reranking
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-12-v2')
pairs = [(query, doc.text) for doc in candidates]
scores = reranker.predict(pairs)
top_20 = sorted(zip(scores, candidates), reverse=True)[:20]

# Stage 3: LLM relevance filter
final = llm_rerank(query, top_20, top_n=5)
```

Impact

NDCG@10: +5.4%

MS MARCO, BEIR

Generation accuracy: +6-8%

FlashRank + Query Expansion

Context tokens: -35%

меньше шума в промпте LLM

Latency: -22%

vs full cross-encoder на всех кандидатах

Query Expansion & Multi-hop Retrieval

Query Expansion

Recall +5-6%

LLM генерирует N вариаций запроса для разных формулировок.

Synonym expansion: climate change

-> global warming, environmental change

HyDE: генерация гипотетического ответа, поиск по его embedding

Multi-Query: параллельный поиск по всем вариациям + merge + dedup

Multi-hop Retrieval

Complex Q&A

Итеративный поиск: ответ на подвопрос используется для следующего поиска.

Q: "Кто основал компанию,

которая купила Twitter?"

Hop 1: "Кто купил Twitter?" -> Elon Musk

Hop 2: "Какие компании основал

Elon Musk?" -> Tesla, SpaceX ...

Synthesize: финальный ответ

```
# Query Expansion + HyDE
def expand_query(query: str, n=3) -> list:
    prompt = f"""
    Generate {n} search queries that
    capture different aspects of: {query}
    Return as JSON list.
    """
    return llm.generate(prompt)

# HyDE: hypothetical doc embedding
def hyde_retrieve(query: str):
    hypo_answer = llm.generate(
        f'Write a passage answering: {query}'
    )
    hypo_embed = embed(hypo_answer)
    return vector_db.search(hypo_embed)
```

```
# Multi-hop retrieval
def multi_hop(query, max_hops=3):
    context = []
    current_q = query
    for hop in range(max_hops):
        docs = retrieve(current_q)
        context.extend(docs)
        # Check if enough info
        if has_answer(context, query):
            break
        # Generate follow-up query
        current_q = llm.generate(
            f'Given: {context}\n'
            f'What else to search for: {query}'
        )
    return synthesize(context, query)
```

Parent Document Retrieval & Hybrid Search

Parent Document Retrieval

Small-to-Large:

Ищем по маленьким чанкам (точность),
возвращаем родительский документ (контекст).

Зачем (Context Precision: +15-25%):

Маленькие чанки -> лучший retrieval
Большие чанки -> лучшая генерация
Parent doc решает оба требования.

```
# Parent Document Retriever
from langchain.retrievers import (
    ParentDocumentRetriever
)
from langchain.storage import InMemoryStore

retriever = ParentDocumentRetriever(
    vectorstore=chroma_db,
    docstore=InMemoryStore(),
    child_splitter=RecursiveCharacter(
        chunk_size=200 # small for search
    ),
    parent_splitter=RecursiveCharacter(
        chunk_size=1000 # large for LLM
    ),
)
```

Hybrid Search (BM25 + Vector)

Reciprocal Rank Fusion (RRF):

Объединяем ранжирования от BM25
(точное совпадение) и vector search
(семантическое сходство).

Зачем:

Vector пропускает ID, коды, имена.
BM25 пропускает синонимы.
Hybrid ловит обе категории.

```
# Hybrid Search with RRF
from rank_bm25 import BM25Okapi

def hybrid_search(query, k=10):
    # Semantic
    vec_results = vector_db.search(
        embed(query), top_k=k*2)
    # Lexical
    bm25_results = bm25.get_top_n(
        query.split(), corpus, n=k*2
    )
    # RRF fusion
    return reciprocal_rank_fusion(
        [vec_results, bm25_results],
        k=60 # RRF constant
    )[:k]
```

Сводная таблица техник

Техника	Тип	Ключевая метрика	Буст	Когда применять
Agentic RAG	Architecture	Answer Correctness	-78% errors	Multi-step, cross-system
GraphRAG	Architecture	Comprehensiveness	+72-83%	Multi-hop, global Q&A
Contextual Retrieval	Indexing	Hit Rate	+15-20%	Документы с cross-ref
Late Chunking	Indexing	Retrieval Accuracy	+10-12%	Long docs, zero LLM cost
Context Compression	Post-retrieval	Token reduction	-35-50%	Большой top-K, шум
Multi-stage Rerank	Post-retrieval	NDCG@10	+5.4%	Всегда в production
Query Expansion	Pre-retrieval	Recall	+5-6%	Нечеткие вопросы, синонимы
Multi-hop	Pre-retrieval	Answer Completeness	High	Complex reasoning chains
Parent Doc Retrieval	Indexing	Context Precision	+15-25%	Структурированные docs
Hybrid Search	Retrieval	Hit Rate + Precision	Baseline	Всегда в production

Roadmap тестирования RAG-техник

Порядок внедрения техник для максимального ROI

1

Baseline

Hybrid Search (BM25 + Vector)

Recursive Chunking

Single-stage Reranking

Метрики: Hit Rate, NDCG

>

2

Optimization

Multi-stage Reranking

Parent Doc Retrieval

Query Expansion / HyDE

Метрики: + Faithfulness, AR

>

3

Advanced

Contextual Retrieval

Context Compression

Late Chunking

Метрики: + Noise Robustness

>

4

Enterprise

Agentic RAG / GraphRAG

Multi-hop Retrieval

Feedback Loop + Monitoring

Метрики: + User Satisfaction

Каждая фаза должна сопровождаться замером метрик из RAG Quality Dashboard. Не переходите к следующей фазе, пока текущие метрики не достигнут пороговых значений. Production baseline 2026: Hybrid Search + Reranking покрывает 80% use cases.

Выводы

- 1 Hybrid Search + Reranking - production baseline для любого RAG в 2026
- 2 Contextual Retrieval - лучшее качество, Late Chunking - лучшая эффективность
- 3 Multi-stage Reranking дает стабильный +5-8% accuracy при снижении latency
- 4 GraphRAG - только для multi-hop / global запросов, не для простых запросов
- 5 Agentic RAG - для cross-system, multi-step задач с автономным планированием
- 6 Каждую технику замеряем: Hit Rate, NDCG, Faithfulness, Noise Robustness, P95 Latency

Метрики качества RAG-систем

Retrieval · Generation · End-to-End · Feedback Loop

Что такое RAG и зачем метрики?

Пайплайн RAG



Группы метрик



Retrieval (3 метрики)

Hit Rate, MRR, NDCG



Generation (4 метрики)

Faithfulness, Relevancy, Correctness, Toxicity



End-to-End (3 метрики)

Context Precision, Recall, F1



Feedback Loop

Мониторинг метрик в продакшене

Метрики качества RAG-систем

MRR (Mean Reciprocal Rank)

0.75+

Средний обратный ранг первого релевантного документа. Показывает, насколько высоко в выдаче находится правильный ответ.

Hit Rate @ K

0.85+

Доля запросов, для которых релевантный документ найден в top-K результатах. Основная метрика recall.

NDCG @ K

0.70+

Нормализованная дисконтированная кумулятивная выгода. Учитывает не только наличие, но и позицию релевантных документов.

Faithfulness (RAGAS)

0.85+

Доля утверждений в ответе LLM, подтвержденных извлеченным контекстом. Метрика галлюцинаций.

Answer Relevancy (RAGAS)

0.80+

Насколько ответ LLM соответствует исходному вопросу пользователя. Оценивается через косинусное сходство.

Context Precision

0.70+

Доля релевантных чанков среди всех извлеченных. Чем выше, тем меньше шума подается в LLM.

Hit Rate @ K

Доля запросов, для которых релевантный документ найден в top-K

Формула

$$\text{Hit Rate @ K} = |\{q : \text{relevant}(q) \cap \text{top-K}(q) \neq \emptyset\}| / |Q|$$

Пример (K=3)

10 запросов, для 8 из них релевантный документ оказался в top-3

$$\text{Hit Rate @ 3} = 8 / 10 = 0.80$$

Ключевое

≥ 0.85

Основная метрика recall для retriever

Низкий Hit Rate →
проблема в эмбедингах, индексе
или стратегии chunking

MRR (Mean Reciprocal Rank)

Средний обратный ранг первого релевантного документа

Формула

$$\text{MRR} = (1/|Q|) \times \sum (1 / \text{rank}_i)$$

rank_i — позиция первого релевантного документа для i-го запроса

Пример

Запрос	Ранг 1-го рел.	1/rank
Q1	1	1.00
Q2	3	0.33
Q3	2	0.50

$$\text{MRR} = (1 + 0.33 + 0.5) / 3 = 0.61$$

Интерпретация

≥ 0.75

MRR = 1.0 — первый результат всегда верный

MRR → 0 — релевантный документ далеко внизу

NDCG @ K

Normalized Discounted Cumulative Gain — учитывает позицию и степень релевантности

Формула

$$DCG @ K = \sum rel_i / \log_2(i + 1)$$

$$NDCG @ K = DCG / IDCG$$

IDCG — DCG идеального ранжирования; $rel_i \in \{0, 1\}$ или градуированная релевантность

Пример (K=4, бинарная релевантность)

≥ 0.70

Результат: [1, 0, 1, 0] $\rightarrow DCG = 1/\log_2(2) + 0 + 1/\log_2(4) + 0 = 1 + 0.5 = 1.5$

Идеал: [1, 1, 0, 0] $\rightarrow IDCG = 1/\log_2(2) + 1/\log_2(3) + 0 + 0 = 1 + 0.63 = 1.63$

$$NDCG @ 4 = 1.5 / 1.63 = 0.92$$

Faithfulness (RAGAS)

Доля утверждений ответа, подтверждённых извлечённым контекстом — метрика галлюцинаций

Формула

$$\text{Faithfulness} = \frac{|\text{подтверждённые утверждения}|}{|\text{все утверждения ответа}|}$$

Пример

Ответ LLM содержит 5 утверждений:

- ✓ «Москва — столица РФ»
- ✓ «Население > 12 млн»
- ✓ «Основана в 1147 г.»
- ✗ «Высочайшее здание — 600 м»
- ✓ «Кремль — объект ЮНЕСКО»

Faithfulness = 4 / 5 = 0.80

Почему критично

≥ 0.85

Низкая Faithfulness →
LLM «выдумывает» факты,
не подтверждённые контекстом

Критично для медицины,
юриспруденции, финансов

Answer Relevancy (RAGAS)

Насколько ответ LLM соответствует исходному вопросу пользователя

Формула

$$AR = (1/N) \times \sum \cos_sim(q, q_i)$$

q — исходный вопрос; q_i — вопрос, сгенерированный LLM из ответа (reverse engineering)

Алгоритм (3 шага)

≥ 0.80

1

Генерация вопросов

LLM генерирует N вопросов из ответа

2

Эмбединги

Получаем эмбединги q и каждого q_i

3

Косинусное сходство

Среднее cos_sim всех пар (q, q_i)

Answer Correctness & Toxicity

Answer Correctness

≥ 0.75

Формула:

$$AC = w_1 \times F1(\text{factual}) + w_2 \times \text{Semantic_sim}$$

Два компонента:

1. **F1 фактуальной точности** — TP, FP, FN утверждений vs ground truth эталон.

TP (ответ LLM=эталон), **FP** (LLM добавила факт, которого нет в эталоне), **FN** (LLM пропустила факт из эталона).

$$\text{Precision} = TP / (TP + FP);$$

$$\text{Recall} = TP / (TP + FN);$$

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}).$$

2. **Семантическое сходство** ответа LLM с ground truth эталоном (cos_sim)

Пример:

$$F1 = 0.8, \text{Sim} = 0.9, w_1=w_2=0.5$$

$$AC = 0.5 \times 0.8 + 0.5 \times 0.9 = 0.85$$

Toxicity / Safety Score

≤ 0.05

Формула:

$$\text{Toxicity} = \max(\text{toxicity_classifier}(\text{answer}))$$

Что оценивает:

- Оскорбления, hate speech
- Вредоносный контент
- Утечка персональных данных

Инструменты:

Detoxify, OpenAI Moderation API, Perspective API, LlamaGuard

Context Precision & Context Recall

Context Precision

≥ 0.70

Формула:

$$CP @ K = (1/R) \times \sum (Precision@k \times rel_k)$$

R — кол-во рел. чанков во всем корпусе

Σ - по кол-ву чанков *K* = 3 (топ-3), *K* = 5 (топ-5)

Precision@k = (кол-во релевантных чанков среди первых *k*) / *k*

rel_k = 1, если чанк на позиции *k* релевантен, иначе 0.

Пример (K=5):

Поз.	1	2	3	4	5
rel	1	0	1	0	1
P@k	1/1	—	2/3	—	3/5

$$CP = (1 + 0.67 + 0.6) / 3 = 0.76$$

CP в отличие от других метрик оценивает сколько нерелевантного контекста попало в ответ LLM

Context Recall

≥ 0.80

Формула:

$$CR = | \text{утверждения GT, подтверждённые контекстом} | / | \text{все утверждения GT} |$$

Что оценивает:

Нашёл ли retriever BCE нужные документы для ответа на вопрос?

Пример:

GT содержит 4 факта, контекст покрывает 3 из них

$$CR = 3 / 4 = 0.75$$

Context F1, Noise Robustness & Latency

Context F1

≥ 0.75

Формула:

$$\text{Context F1} = 2 \times \text{CP} \times \text{CR} / (\text{CP} + \text{CR})$$

Гармоническое среднее

Context Precision и

Context Recall

(Context F1 штрафует за дисбаланс)

Пример:

CP=0.76, CR=0.75

$$\text{F1} = 2 \times 0.76 \times 0.75 / (0.76 + 0.75)$$

$$\text{F1} = 0.755$$

Noise Robustness

≥ 0.90

Формула:

$$\text{NR} = \frac{\text{Score}(\text{с шумом})}{\text{Score}(\text{без шума})}$$

Устойчивость ответа

при добавлении

нерелевантных чанков

в контекст

(Score - любая из рассмотренных

метрик LLM; например,

Answer Correctness)

Зачем:

1. Сравнение разных LLM на устойчивость к шуму
2. Эксперименты с промптами “игнорируй нерелевантное”

Latency (P50/P95)

Формулы:

P50 = median(t)

P95 = percentile_95(t)

TTFT = time to first token

P50 - типичная задержка

P95 - задержка тяжелых запросов

Целевые:

Retrieval: < 200ms

E2E (P95): < 3s

TTFT: < 500ms

Не забывайте: быстрый
но неточный RAG бесполезен

Сводная таблица всех метрик

Метрика	Группа	Цель	Что измеряет
Hit Rate @ K	Retrieval	≥ 0.85	Recall: найден ли документ в top-K
MRR	Retrieval	≥ 0.75	Позиция первого релевантного
NDCG @ K	Retrieval	≥ 0.70	Качество ранжирования
Faithfulness	Generation	≥ 0.85	Доля подтверждённых утверждений
Answer Relevancy	Generation	≥ 0.80	Соответствие ответа вопросу
Answer Correctness	Generation	≥ 0.75	F1 + семантическое сходство с GT
Toxicity	Generation	≤ 0.05	Безопасность ответа
Context Precision	End-to-End	≥ 0.70	Доля релевантных чанков
Context Recall	End-to-End	≥ 0.80	Покрытие всех нужных документов
Context F1	End-to-End	≥ 0.75	Баланс Precision и Recall
Noise Robustness	End-to-End	≥ 0.90	Устойчивость к шуму в контексте
Latency P95	Операц.	$< 3s$	Время ответа (end-to-end)

Как замерять метрики в продакшене

Замкнутый цикл: логирование → оценка → анализ → улучшение → повтор



1. Логирование

Сохраняем каждый запрос, извлечённые чанки, ответ LLM, user feedback



2. Оценка

Автоматическое вычисление метрик по RAGAS / LLM-judge + пользовательские оценки



3. Мониторинг

Дашборды, алерты при падении метрик ниже пороговых значений



4. Улучшение

Анализ провалов, A/B тесты, обновление эмбеддингов, промптов, chunking

🔄 Непрерывный цикл: результаты шага 4 → повторный замер (шаг 2) → отслеживание улучшений

Шаг 1: Логирование каждого запроса

Что логировать

Для каждого запроса сохраняем:

query — исходный вопрос
retrieved_chunks[] — чанки + scores
reranked_chunks[] — после reranker
prompt — финальный промпт в LLM
answer — ответ LLM
latency_ms — время каждого этапа
user_feedback — лайк/дизлайк, коррекция
timestamp, user_id, session_id
model_version, config_version

Инструменты логирования

Trace-платформы:

LangSmith, LangFuse, Arize Phoenix,
Weights & Biases, Helicone

Self-hosted:

OpenTelemetry + PostgreSQL / ClickHouse

User feedback:

- Thumbs up/down в UI
- «Помог ли ответ?» опрос
- Копирование ответа = implicit +
- Повторный запрос = implicit –

Шаг 2: Автоматическая оценка

Online-оценка (в реальном времени)

Без ground truth — что можно считать:

Retrieval:

- Cosine similarity (query $\leftrightarrow$ chunks)
- Retriever confidence scores

Generation:

- Faithfulness (LLM-as-judge)
- Answer Relevancy
- Toxicity (classifier)
- User satisfaction (thumbs)

Latency:

- P50, P95, TTFT — каждый запрос

Offline-оценка (периодическая)

С ground truth — полный набор:

Тестовый датасет:

50-200 вопросов с эталонными ответами, покрывающих основные сценарии использования

Метрики:

Hit Rate, MRR, NDCG (retrieval)
Answer Correctness (generation)
Context Recall, Context F1 (E2E)

Частота: при каждом изменении

модели, промпта или индекса

Шаг 3: Мониторинг и алерты

Дашборд метрик

Faithfulness (скользящее среднее за 24ч)
Answer Relevancy (rolling avg)
Hit Rate @ K (по дням)
Latency P95 (по часам)
User satisfaction rate (%)
Toxicity alerts (count)

Правила алертинга

Faithfulness < 0.80 → CRITICAL
Hit Rate < 0.75 → WARNING
Latency P95 > 5s → WARNING
Toxicity > 0.1 → CRITICAL
User dislike rate > 20% → WARNING

Стек мониторинга

Grafana + Prometheus, Superset

Дашборды, алерты,
исторические тренды

LangFuse / LangSmith

LLM-специфичный
мониторинг + tracing

E-mail / Messenger

Уведомления при
срабатывании алертов

Шаг 4: Анализ провалов и улучшение

Анализ провалов

1. Фильтруем запросы с дизлайками или низкими метриками

2. Классифицируем причину:

Retrieval failure — документ не найден

Ranking failure — документ найден,
но слишком низко

Generation failure — контекст ОК,
но LLM ответил плохо

Missing knowledge — данных нет в базе

3. Приоритизируем по частоте и бизнес-импакту

А/В тестирование

Что тестировать А/В:

- Embedding модель (e5 vs BGE vs OpenAI)
- Chunk size (256 vs 512 vs 1024)
- Chunk overlap (0 vs 64 vs 128)
- Reranker (без vs cross-encoder)
- Top-K (3 vs 5 vs 10)
- LLM модель и промпт
- Гибридный vs dense поиск

Протокол:

1. Фиксируем eval dataset
2. Меняем ОДИН параметр
3. Сравниваем ВСЕ метрики

Как замерять каждую метрику в продакшене

Метрика	Как замерять в продакшене
Hit Rate / MRR / NDCG	Offline eval: 50+ тестовых вопросов с размеченными рел. документами, запуск при каждом изменении индекса
Faithfulness	Online: LLM-judge проверяет каждый ответ vs извлечённые чанки (async pipeline)
Answer Relevancy	Online: LLM генерирует вопросы из ответа, считает cos_sim с оригиналом (batch каждые 15 мин)
Answer Correctness	Offline: сравнение с ground truth на eval dataset, при обновлении модели / промпта
Toxicity	Online: real-time classifier (Detoxify) на каждый ответ, блокировка при score > threshold
Context Precision/Recall	Offline eval + online proxy: отношение retrieved scores > threshold к total retrieved
Latency	Online: OpenTelemetry трейсы, P50/P95 агрегация, алерт при превышении SLA
User Satisfaction	Online: thumbs up/down, implicit signals (копирование, повтор запроса), NPS опросы

Практические рекомендации

1 Начните с Hit Rate

Если документы не находятся,
остальные метрики бессмысленны

2 Мониторьте Faithfulness

Галлюцинации — главная
проблема RAG в продакшене

3 Создайте eval dataset

50-200 вопросов с GT — основа
для offline оценки

4 Автоматизируйте с RAGAS

`pip install ragas` — полный
набор метрик из коробки

5 Стройте feedback loop

Логирование → оценка →
мониторинг → улучшение

6 Не забудьте latency

Без SLA на скорость
пользователи уйдут

RAG Quality Dashboard (Superset / Grafana)

Как организовать единый дашборд мониторинга RAG-системы в продакшене

Структура дашборда — 6 панелей

1. Retrieval Quality

Line chart: Hit Rate, MRR,
NDCG за 7/30 дней
Порог: зелёная линия

2. Generation Quality

Line chart: Faithfulness,
Answer Relevancy, Toxicity
Rolling average 24ч

3. User Satisfaction

Bar chart: % thumbs up/down
по дням + NPS score
Target: > 80% positive

4. Latency Distribution

Histogram + P50/P95 линии
Retrieval vs Generation
SLA: P95 < 3s

5. Failure Analysis

Heatmap ошибок по типу:
retrieval / ranking /
generation / missing KB

6. KB Coverage

Pie chart: % запросов
покрытых базой знаний
Тренд по неделям

Пороговые значения и визуальные индикаторы

Пороги для Superset / Grafana

Метрика	OK	WARNING	CRITICAL	Тип графика
Faithfulness	> 0.85	0.70-0.85	< 0.70	Line + threshold
Hit Rate @ K	> 0.85	0.70-0.85	< 0.70	Line + threshold
Answer Relevancy	> 0.80	0.65-0.80	< 0.65	Line + threshold
MRR	> 0.75	0.60-0.75	< 0.60	Gauge
Latency P95	< 3s	3-5s	> 5s	Histogram
Toxicity	< 0.05	0.05-0.10	> 0.10	Counter + alert
User like rate	> 80%	60-80%	< 60%	Bar chart
KB Coverage	> 90%	75-90%	< 75%	Pie + trend

Визуальные правила

В Superset / Grafana:

Зелёная зона — всё OK, метрики выше порога

Жёлтая зона — WARNING, уведомление в Slack

Красная зона — CRITICAL, PagerDuty + блокировка

Паттерн из LLM Under Hood:

heatmap ошибок по полям × документам для приоритизации исправлений

От дашборда к улучшению RAG и базы знаний

Сигнал на дашборде → Действие

Hit Rate падает ↓

→ Обновить эмбединги / добавить документы в базу знаний / пересмотреть chunking стратегию (размер, overlap)

Faithfulness падает ↓

→ Проверить prompt / добавить guardrails / уменьшить температуру / reranker

KB Coverage падает ↓

→ Анализ «пустых» запросов — добавить недостающие документы в базу

Latency растёт ↑

→ Оптимизировать индекс / кэширование

Цикл улучшения базы знаний

1. Собираем запросы с низким Hit Rate
→ кластеризуем по темам
2. Определяем «пробелы» в KB —
темы без покрытия документами
3. Создаём / обновляем документы
(Domain Expert review — DDD подход)
4. Переиндексируем + offline eval
на тестовом датасете
5. Сравниваем метрики до/после
на дашборде (A/B тест)

Query Expansion + Company Context

помогают поднять Hit Rate на 15-30%

Airflow: расписание обновления данных

Airflow DAGs для RAG мониторинга

DAG 1: rag_online_metrics (каждые 15 мин)

- Считает Faithfulness, Relevancy по новым запросам (LLM-as-judge)
- Агрегирует в PostgreSQL / ClickHouse

DAG 2: rag_offline_eval (ежедневно 03:00)

- Прогоняет eval dataset (50-200 Q) через RAGAS: Hit Rate, MRR, NDCG, Context Recall, Answer Correctness

DAG 3: kb_reindex (при изменении KB)

- Trigger: webhook / schedule еженедельно
- Переиндексация + автоматический запуск DAG 2 для сравнения

Частота обновления данных

Данные	Частота	Причина
Логи запросов	Real-time	Streaming в ClickHouse
Online метрики	15 мин	LLM-judge batch
Offline eval	1×/день	Полный eval dataset
Toxicity check	Real-time	Classifier на каждый ответ
User feedback	Real-time	Событийная архитектура
KB reindex	1×/неделю	Или при изменении KB
Дашборд Superset	5-15 мин	Auto-refresh Superset
Алерты Grafana	1 мин	Prometheus scraping

Token Caching (из курса LLM Under Hood):

Для offline eval с Checklists — 30-70% быстрее и 50-90% дешевле

Бизнес-метрики RAG-систем

Реальные результаты внедрения ИИ-суфлера на основе RAG

+21%

Скорость обработки
обращений по всем каналам

x2

Продуктивность
каналов продаж за 2025

+5%

Удовлетворенность
клиентов (CSI)

6-8 нед

Период адаптации
сотрудника (было 12)

-31%

Обоснованные жалобы
на консультации

+5%

Решено с первого
обращения (FCR)

1 000+

Статей переработано
командой проекта

+18.6%

Удобство восприятия
статей (переупаковка)

+28.4%

Качество обработки
обратной связи

ROI (return of investments) RAG-системы

$$ROI = (Savings + Revenue) - (Infra + Tokens + FTE + Maintenance)$$

Savings = (Avg Handle Time) x (Volume) x (Cost/min)

Revenue = FCR uplift x Retention x LTV

Cost = Infra + Tokens + FTE + Maintenance

PnL (Profit and Loss) калькулятор (50 операторов, 10K обращений/мес)

	Статья	Мес.	Год
+	AHT savings (-21% x 50 ops)	525K	6.3M
+	Onboarding savings (12->7 нед)	150K	1.8M
+	FCR uplift (+5% retention)	200K	2.4M
-	Infra (GPU/VDB/hosting)	80K	960K
-	LLM tokens (10K req x 4K tok)	45K	540K
-	FTE (2 ML eng + 0.5 PM)	625K	7.5M
=	Net Impact (ROI ~58%)	125K	1.5M

Фреймворки оценки RAG-систем

RAGAS · Arize Phoenix · RAGChecker · LangFuse · LangSmith · DeepEval · TruLens

Как выбрать и использовать в продакшн RAG-проекте

Ландшафт инструментов RAG Evaluation

Два типа инструментов: Evaluation (оценка качества) и Observability (мониторинг в продакшене)

Evaluation — оценка качества

RAGAS

Open Source

LLM-based метрики, синтетические тесты, интеграции с LangChain/LlamaIndex

RAGChecker

Open Source

Claim-level диагностика retrieval + generation от Amazon Science (NeurIPS 2024)

DeepEval

Open Source

Pytest-стиль eval, CI/CD интеграция, red-teaming, 14+ метрик RAG

TruLens

Open Source

RAG Triad (context relevance, groundedness, answer relevance), feedback functions

Observability — мониторинг и tracing

LangFuse

Open Source

Open-source tracing, prompt mgmt, eval scores, datasets, self-hosted option

LangSmith

Commercial

Tracing + eval от LangChain, playground, datasets, CI/CD, Hub

Arize Phoenix

Open Source

OpenTelemetry-native, tracing, LLM evals, prompt playground, 8k+ stars

Helicone

Open Source

Gateway-proxy: cost tracking, rate limits, caching, logging, analytics

RAGAS — стандарт де-факто

github.com/explodinggradients/ragas — Open Source, LLM-based оценка без ground truth

Встроенные метрики

Retrieval:

- Context Precision — доля релевантных чанков
- Context Recall — покрытие GT контекстом
- Context Entity Recall — сущности из GT

Generation:

- Faithfulness — подтверждённость контекстом
- Answer Relevancy — соответствие вопросу
- Answer Correctness — F1 + sem. similarity
- Noise Sensitivity — устойчивость к шуму

Дополнительно:

- Синтетическая генерация тестов
- Интеграция с LangChain, LlamaIndex, Phoenix

Как использовать в продакшене

1. Offline eval (ежедневно / при изменении):

```
from ragas import evaluate
result = evaluate(dataset, metrics)
```

2. Синтетический eval dataset:

```
from ragas.testset import TestsetGenerator
Автогенерация Q&A из ваших документов
```

3. CI/CD интеграция:

Запуск eval в GitHub Actions / GitLab CI
при изменении prompt / модели / индекса

Стоимость: ~\$0.05-0.20 за запрос (GPT-4o judge)

Можно использовать локальные модели

RAGChecker & DeepEval

RAGChecker (Amazon Science)

NeurIPS '24

Что это:

Fine-grained диагностика на уровне claims (утверждений), а не целых ответов

Метрики:

- Claim-level Precision / Recall
- Faithfulness, Hallucination Rate
- Context Utilization — сколько контекста реально использовано в ответе
- Noise Sensitivity — диагностика шума

Когда использовать:

Глубокий аудит системы, сравнение RAG конфигураций, исследование. Интеграция с LlamaIndex из коробки.

DeepEval

CI/CD Ready

Что это:

Pytest-style тестирование LLM-приложений с фокусом на RAG и hallucination detection

Ключевые фичи:

- 14+ RAG-метрик из коробки
- deepeval test run — как pytest
- Синтетическая генерация тестов
- Red-teaming (jailbreak, bias, toxicity)
- GitHub Actions / GitLab CI интеграция

Когда использовать:

Команды, привыкшие к unit-тестам. Регрессионное тестирование RAG в CI/CD пайплайне.

LangFuse & LangSmith

LangFuse — Open Source Observability

Фичи:

- Трейсинг каждого LLM-вызова (spans)
- Prompt management + versioning
- Eval scores на трейсы (custom + LLM)
- Datasets для offline eval
- Self-hosted (Docker) или Cloud
- Интеграции: LangChain, LlamaIndex, OpenAI SDK, Anthropic, Haystack

В продакшене:

Основной инструмент для логирования и observability. Рекомендуется как основа стека.

LangSmith — LangChain Ecosystem

Commercial

Фичи:

- Tracing LangChain пайплайнов
- Prompt playground + A/B тесты
- Datasets + eval runners
- Online eval (LLM-as-judge на трейсы)
- Hub — sharing промптов и цепочек
- Annotation queues для human review

Когда выбирать:

Если проект на LangChain — естественный выбор. Лучший DX для LangChain стека.

Минусы:

Vendor lock-in, коммерческая модель, нет self-hosted варианта

Arize Phoenix & TruLens

Arize Phoenix — OTel-native LLM Obs

8k+ Stars

Ключевое:

- OpenTelemetry-native — интеграция в существующий стек (Grafana, Jaeger)
- LLM evaluators + code-based метрики
- Prompt playground для экспериментов
- Human annotation workflows
- Auto-инструментация: LangChain, LlamaIndex, DSPy, OpenAI, Anthropic

В продакшене:

Идеален если уже есть OTel-стек.
Локальный запуск для dev, Phoenix Cloud для prod. Хорошо дружит с RAGAS.

TruLens — RAG Triad Evaluation

RAG Triad (3 ключевые метрики):

1. Context Relevance

Насколько контекст релевантен вопросу

2. Groundedness

Подтверждён ли ответ контекстом

3. Answer Relevance

Отвечает ли ответ на вопрос

Когда использовать:

Быстрый старт с минимумом кода.
Хорош для прототипов и R&D.
OpenTelemetry + version comparison.

Сравнение фреймворков

Фреймворк	Тип	Self-hosted	Метрики RAG	Tracing	CI/CD	Рекомендация
RAGAS	Eval	—	8+	—	Да	Must-have для offline eval
RAGChecker	Eval	—	Claim-level	—	—	Глубокий аудит
DeepEval	Eval	—	14+	—	Да	Регрессионные тесты (когда поменяли: модель, промпт и тд и хотим сравнить с тем что было до)
TruLens	Eval+Obs	Да	RAG Triad	Да	—	Быстрый старт
LangFuse	Obs	Да	Custom	Да	—	Основа для prod
LangSmith	Obs	Нет	Custom	Да	Да	LangChain проекты
Arize Phoenix	Obs	Да	LLM evals	OTel	—	OTel-стек команды

Рекомендация: RAGAS (offline eval) + LangFuse (prod tracing) + DeepEval (CI/CD) — минимальный стек для продакшн RAG. Добавьте RAGChecker для глубокого аудита и Arize Phoenix если уже используете OpenTelemetry.

Рекомендуемый стек для продакшн RAG



Tracing & Logging

LangFuse (self-hosted) или Arize Phoenix (OTel). Каждый запрос → трейс с spans

langfuse / phoenix



Online Evaluation

LLM-as-judge на каждый ответ: Faithfulness, Relevancy, Toxicity (async)

ragas + langfuse scores



Offline Eval & CI/CD

RAGAS eval dataset + DeepEval в CI/CD при каждом PR / deploy

ragas + deepeval



Deep Diagnostics

RAGChecker для аудита при падении метрик. Claim-level анализ причин

ragchecker

Порядок внедрения

Неделя 1: LangFuse tracing → **Неделя 2:** RAGAS offline eval + dataset → **Неделя 3:** DeepEval CI/CD → **Неделя 4:** Online eval + дашборд + алерты

Фреймворки Open Source RAG-систем

RAGFlow · Dify

Как выбрать и использовать в продакшн RAG-проекте

Зачем нужны коробочные RAG-решения



Быстрый старт

Развертывание за часы вместо недель. Docker Compose, веб-интерфейс из коробки, минимум кода.



Встроенные техники

Перанжирование, гибридный поиск, GraphRAG, RAPTOR и другие техники доступны без ручной реализации.



Локальное развертывание

Полная изоляция данных. Ollama, vLLM для локальных LLM. Никакого трафика за периметр.



Без DevOps-экспертизы

Управление знаниями через UI. Загрузка документов, настройка чанкинга, тестирование поиска.

Ключевые Open Source RAG-платформы

Полностью локальное развертывание через Docker



RAGFlow

35k+ GitHub

Глубокий парсинг документов, GraphRAG, RAPTOR, реранкинг, визуальный редактор пайплайнов



Dify

60k+ GitHub

Low-code платформа с RAG, агентами, управление моделями. Визуальный конструктор workflow



AnythingLLM

35k+ GitHub

Простой desktop/server RAG с агентами, множество векторных БД, гибкие настройки



Kotaemon

18k+ GitHub

Настраиваемый RAG UI для конечных пользователей и разработчиков, multi-modal



Haystack

20k+ GitHub

Enterprise-ready NLP пайплайны, компонентная архитектура, мониторинг и A/B тесты



LLMWare

12k+ GitHub

Легковесный RAG на малых моделях, **работает на CPU**, минимальные требования к ресурсам

RAGFlow

SOTA коробочное RAG-решение

Глубокий парсинг документов (OCR)

Распознавание таблиц, изображений, формул из PDF/DOCX/XLSX. OCR сканированных документов. Сохранение структуры и семантики.

Шаблоны чанкинга

Шаблоны для разных типов данных: FAQ, книги, юридические документы, таблицы. Визуализация чанкинга с ручной корректировкой.

Гибридный поиск и Реранкинг

BM25 + векторный поиск. Fused re-ranking. Поддержка cross-encoder и tensor-based reranking моделей.

GraphRAG + RAPTOR

Построение Knowledge Graph из документов. Иерархическая кластеризация RAPTOR для multi-hop вопросов.

TreeRAG

Двухуровневый поиск: Search (точный recall мелкими фрагментами) и Retrieve (динамическая сборка контекста через дерево).

Агентный фреймворк

Визуальный DAG-редактор для построения агентов с памятью, multi-step reasoning, вызовом инструментов.

RAGFlow: архитектура и развертывание



Загрузка
документов



Deep
Parsing



Chunking +
Enrichment



Embedding +
Indexing



Hybrid
Retrieval



Re-ranking +
Generation

Развертывание

Docker Compose, 2 GB (slim) или 9 GB (full с моделями). Поддержка GPU для DeepDoc. Elasticsearch / Infinity для хранения.

Локальные LLM

Ollama, vLLM, Xinference. Любая модель через OpenAI-совместимый API. Полная изоляция данных.

Интеграции

REST API, Python SDK. Langfuse для observability. Поддержка S3/MinIO для хранения файлов.

Сравнение коробочных Open Source RAG-платформ

Платформа	Качество парсинга	Гибридный поиск	Реранкинг	Graph RAG	Агенты	UX/UI	Локальные LLM	API	Сложность
RAGFlow	++++	+++	+++	+++	+++	+++	+++	+++	Средняя
Dify	++	++	+	--	++++	++++	+++	++++	Низкая
AnythingLLM	+	+	--	--	++	++	+++	++	Низкая
Kotaemon	++	++	++	+	+	++	++	+	Средняя
Haystack	++	+++	+++	++	++	+	+++	++++	Высокая
LLMWare	+	++	+	--	+	+	+++	++	Низкая

++++ = отлично +++ = хорошо ++ = базово + = минимально -- = отсутствует

RAGFlow обеспечивают наивысшее качество поиска среди коробочных решений за счет качества парсинга документов, шаблонного чанкинга, TreeRAG, GraphRAG и встроенного реранкера.

Но эмбединги, реранкер и парсеры в коробочных решениях заточены под латиницу => с кириллицей качество будет хуже.

Коробочные решения vs Ручная разработка RAG

Критерий	Коробочные решения (RAGFlow, Dify)	Ручная разработка (LangChain, LlamaIndex)
Время до MVP	Часы / дни	Недели / месяцы
Качество парсинга	Высокое для латиницы	Зависит от усилий
Качество поиска (MRR)	0.50 - 0.70	0.70 - 0.90+
Гибкость настройки	Ограничена UI/API	Полная
Кастомные техники	Сложно добавить	Любые эксперименты
Поддержка и DevOps	1 DS	1 DS
Масштабирование	Docker / K8s готово	Сборка кастомного образа
Метрики и мониторинг	Langfuse, UI дашборды	Arize Phoenix

Коробочные решения оптимальны для быстрого запуска и стандартных задач.
Ручная разработка дает максимальное качество.

Выводы

Для быстрого запуска (MRR 0.7)

RAGFlow: лучший баланс качества поиска, глубины парсинга и готовности к production. Развертывание за 1 день.

Для low-code AI-приложений с агентами (MRR 0.6)

Dify: визуальный конструктор workflow, управление моделями, RAG + агенты в одном интерфейсе.

Для максимального качества поиска (MRR 0.8+)

Ручная разработка: semantic chunking + retrieval + cross-encoder reranking + contextual compression.

Инструменты оценки качества RAG

RAGAS, DeepEval

Инструменты сбора обратной связи RAG

LangSmith, Phoenix by Arize

SOTA RAG-техники: обзор и результаты

Техника	Описание	LLM	Сложн.	Эффект	Статус / Источник
Semantic Chunking	Разбиение по семантическим границам	Да	1	Да	Улучшение MRR
Fusion Retrieval	BM25 + vector search ensemble	Нет	1	Да	BlendedRAG, IBM 2024
Cross-Encoder Reranking	Повторное ранжирование cross-encoder	Нет	1	Да	Значительное улучшение
Contextual Compression	Суммаризация документов для поиска	Да	2	Да	Значительный прирост качества
Context Window	Добавление N соседних чанков	Нет	2	Да	Решает разрывы контекста
Late Chunking	Чанкинг после embedding (Jina 2024)	Нет	2	Да	ECIR 2025 paper
Contextual Retrieval	LLM-обогащение чанков контекстом	Да	3	Да	лучшая когерентность
TreeRAG	Иерархич. навигация по дереву документа	Да	4	Да	ACL 2025, RAGFlow
GraphRAG	Knowledge Graph для multi-hop QA	Да	5	Mixed	Microsoft 2024
Query Transform	Переписывание / декомпозиция запроса	Да	2	Нет	Галлюцинации ухудшают поиск
RAPTOR	Иерарх. кластеризация + суммаризация	Да	4	Нет	Долгая обработка, не масштаб.
HyDE	Гипотетический документ как запрос	Да	3	Нет	Генерирует шум для узких доменов

Современные подходы и исследования

Context Engineering

Переход от оптимизации отдельных RAG-алгоритмов к проектированию полного пайплайна retrieval - context assembly - reasoning. RAGFlow 2025 year-end review.

Dual-Granularity RAG (Search + Retrieve)

Разделение на мелкозернистый поиск (recall) и динамическую сборку крупного контекста (comprehension). TreeRAG, ACL Findings 2025.

Tensor-Based Reranking (ColBERT v2)

Хранение token-level embeddings, попарное сравнение с запросом. Качество cross-encoder при стоимости bi-encoder. JaColBERT, Jina-ColBERT-v2.

NodeRAG

Гетерогенный граф: превосходит GraphRAG и LightRAG по скорости индексации, времени запроса, качеству multi-hop QA. Hugging Face Papers 2025.

Multimodal RAG + VLM

Поиск по изображениям внутри PDF с помощью ColPali / Vision-Language Models. RAGFlow поддерживает multimodal retrieval.

Late Chunking (Jina AI)

Embedding всего документа до чанкинга сохраняет глобальный контекст. Эффективнее naive chunking, дешевле ColBERT. ECIR 2025 Workshop.

10 RAG-архитектур 2026

По Gartner: >70% enterprise GenAI-инициатив к 2026 потребуют structured retrieval pipelines

01 Hybrid RAG

BM25 + semantic + reranker

02 GraphRAG

Knowledge graphs + multi-hop

03 Agentic RAG

Autonomous plan-retrieve-reason

04 Multimodal RAG

Images, audio, tables, video

05 Adaptive RAG

Cost-optimized by complexity

06 Self-Correcting RAG

Evaluate -> re-query loops

07 Conversational RAG

Multi-turn context memory

08 Federated RAG

Cross-system retrieval

09 Streaming RAG

Real-time data feeds

10 RAG + Long Context

Hybrid with 1M+ token windows

Куда углубиться в RAG research?

GraphRAG + Knowledge Engineering

Самый востребованный навык по Gartner. Построение knowledge graphs, multi-hop reasoning, entity extraction. Мало специалистов - высокий спрос.

Agentic RAG Orchestration

Проектирование автономных агентов: planning, tool use, self-correction. Frameworks: LangGraph, LlamaIndex, CrewAI. Ключевая роль в 2026-2028.

RAG Evaluation & Observability

RAGAS, BEIR-метрики, faithfulness scoring. RAGOps -- новый MLOps. Критически мало экспертов, огромный спрос в enterprise.

Context Engineering

Gartner D&A Summit 2026: context graphs дают 5x рост точности AI-аналитиков. Semantic layers, metadata governance, access control.

Спасибо за внимание

Пайплайн RAG / Метрики / 10+ техник / Agentic & Graph RAG / Коробочные решения / Production

8

Базовых концептов

10+

RAG-техник

12

метрик

Colab

Практики

RAG: архитектура, метрики, production